Casos de uso sobre la arquitectura integrada JavaEE · .NET

SSO · Subastas en línea · 10 casos de uso · 214 pasos · número de paso · origen → destino · mensaje

`m` camino desde la app móvil
`a` / `b` / `c` caminos alternos
`r1` y `r2` misma flecha en las dos réplicas de MSCompra

I Autenticar cliente
II Consultar catálogo y suscribirse
III Publicar y abrir subasta
IV Registrar oferta y avisar
V Cerrar subasta y crear compra
VI Iniciar compra y validar OFAC
VII Pagar en YoPago (.NET)
VIII Despachar con SIN y SEN
IX Notificar al cliente
X Compensar la compra

I Autenticar cliente

DS01 · Verde · RGB 0, 153, 0

I.1	Browser → PantallaWebSubasta	`ingresarCredenciales(usuario, clave)`
I.2	PantallaWebSubasta → ProxyWSSubastaWeb	`autenticar(usuario, clave)`
I.2m	PantallaMobileSubasta → ProxyWSSubastaMobile	`autenticar(usuario, clave)`
I.3	ProxyWSSubastaWeb → Nginx	`POST /api/acceso/login`
I.3m	ProxyWSSubastaMobile → Nginx	`POST /api/acceso/login`
I.4	Nginx → WSGateway	`autenticar(usuario, clave)`
I.5	WSGateway → SrvSeguridadToken	`evaluarAcceso(rutaLogin)`
I.6	SrvSeguridadToken → SrvRuteo	`enrutarLogin(usuario, clave)`
I.7	SrvRuteo → WSAcceso	`autenticar(usuario, clave)`
I.8	WSAcceso → SrvLogin	`autenticar(usuario, clave)`
I.9	SrvLogin → IntegraLoginAD	`validarCredenciales(usuario, clave)`
I.10	IntegraLoginAD → AD	`autenticar(usuario, clave)` LDAP
I.11	SrvLogin → RepoLogin	`obtenerDatosCliente(identidadValidada)`

II Consultar catálogo y suscribirse

DS02, DS03-A, DS03-B · Naranja · RGB 255, 128, 0

II.1	Browser → PantallaWebSubasta	`consultarCatalogo(filtros) / suscribirse(criterio)`
II.2	PantallaWebSubasta → ProxyWSSubastaWeb	`consultarCatalogo / suscribirse (token)`
II.2m	PantallaMobileSubasta → ProxyWSSubastaMobile	`consultarCatalogo / suscribirse (token)`
II.3	ProxyWSSubastaWeb → Nginx	`GET /api/catalogo · POST /api/suscripciones`
II.3m	ProxyWSSubastaMobile → Nginx	`GET /api/catalogo · POST /api/suscripciones`
II.4	Nginx → WSGateway	`consultarCatalogo / suscribirse`
II.5	WSGateway → SrvSeguridadToken	`validarYAutorizar(solicitud)`
II.6	SrvSeguridadToken → SrvRuteo	`enrutarConsultaCatalogo / enrutarSuscripcion`
II.7a	SrvRuteo → WSCatalogo	`consultarCatalogo(filtros)` a: catálogo
II.8a	WSCatalogo → SrvCatalogo	`consultarCatalogo(filtros)`
II.9a	SrvCatalogo → RepoCatalogo	`buscarSubastasPublicadas(filtros)`
II.7b	SrvRuteo → WSSuscripcion	`suscribirse / cancelarSuscripcion` b: suscripción
II.8b	WSSuscripcion → SrvSuscripcion	`suscribirse / cancelarSuscripcion`
II.9b	SrvSuscripcion → RepoSuscripcion	`activarSuscripcion / cancelarSuscripcion`

III Publicar y abrir subasta

DS07, DS08, DS10, DS11-A, DS11-B · Morado · RGB 128, 0, 255

III.1	Browser → PantallaWebSubasta	`solicitarPublicacion(idSubasta)` administrador
III.2	PantallaWebSubasta → ProxyWSSubastaWeb	`publicarNuevaSubasta(idSubasta, token)`
III.2m	PantallaMobileSubasta → ProxyWSSubastaMobile	`publicarNuevaSubasta(idSubasta, token)`
III.3	ProxyWSSubastaWeb → Nginx	`POST /api/catalogo/publicar`
III.3m	ProxyWSSubastaMobile → Nginx	`POST /api/catalogo/publicar`
III.4	Nginx → WSGateway	`publicarNuevaSubasta(idSubasta, token)`

III.5	WSGateway → SrvSeguridadToken	`validarYAutorizar(solicitudPublicacion)`
III.6	SrvSeguridadToken → SrvRuteo	`enrutarPublicacion(idSubasta, admin)`
III.7	SrvRuteo → WSCatalogo	`publicarNuevaSubasta(idSubasta, admin)`
III.8	WSCatalogo → SrvCatalogo	`publicarNuevaSubasta(idSubasta, admin)`
III.9	SrvCatalogo → RepoCatalogo	`publicarSiPendienteYEnFecha(idSubasta)`
III.10	SrvCatalogo → RepoOutboxCatalogo	`guardar(NuevaSubasta)`
III.11	WorkerOutboxCatalogo → RepoOutboxCatalogo	`obtenerPendientes(tamañoLote)`
III.12	WorkerOutboxCatalogo → q.nuevaSubasta	`publicar(NuevaSubasta)` AMQP
III.13	TimerAperturaSubasta → SrvSubasta	`abrirSubastaProgramada(idSubasta, fechaActual)`
III.14	SrvSubasta → RepoSubasta	`abrirSiProgramadaYEnPlazo(idSubasta)`
III.15	SrvSubasta → RepoOutboxSubasta	`guardar(SubastaIniciada)`
III.16	WorkerOutboxSubasta → RepoOutboxSubasta	`obtenerPendientes(tamañoLote)`
III.17	WorkerOutboxSubasta → q.subastaIniciada	`publicar(SubastaIniciada)` AMQP
III.18a	q.nuevaSubasta → ListenerEventoSubasta	`entregar(evento)` a: nueva subasta
III.18b	q.subastaIniciada → ListenerEventoSubasta	`entregar(evento)` b: subasta iniciada
III.19	ListenerEventoSubasta → SrvSuscripcion	`procesarEvento(evento)`
III.20	SrvSuscripcion → RepoSuscripcion	`yaProcesado(idEvento)` · `buscarSuscriptores()`
III.21	SrvSuscripcion → RepoOutboxSuscripcion	`guardar(Notificacion)`
III.22	WorkerOutboxSuscripcion → RepoOutboxSuscripcion	`obtenerPendientes(tamañoLote)`
III.23	WorkerOutboxSuscripcion → q.notificacion	`publicar(Notificacion)` → IX

IV — Registrar oferta y avisar

DS04-A, DS04-B, DS11-C

Fucsia · RGB 230, 0, 126

IV.1	Browser → PantallaWebSubasta	`registrarOferta(idSubasta, valor)`
IV.2	PantallaWebSubasta → ProxyWSSubastaWeb	`registrarOferta(idSubasta, valor, token)`
IV.2m	PantallaMobileSubasta → ProxyWSSubastaMobile	`registrarOferta(idSubasta, valor, token)`
IV.3	ProxyWSSubastaWeb → Nginx	`POST /api/subastas/{id}/ofertas`
IV.3m	ProxyWSSubastaMobile → Nginx	`POST /api/subastas/{id}/ofertas`
IV.4	Nginx → WSGateway	`registrarOferta(idSubasta, valor, token)`
IV.5	WSGateway → SrvSeguridadToken	`validarYAutorizar(solicitud)`
IV.6	SrvSeguridadToken → SrvRuteo	`enrutarOferta(idSubasta, cliente, valor)`
IV.7	SrvRuteo → WSSubasta	`registrarOferta(idSubasta, cliente, valor)`
IV.8	WSSubasta → SrvSubasta	`registrarOferta(idSubasta, cliente, valor)`
IV.9	SrvSubasta → RepoSubasta	`obtenerEstadoYVersion(idSubasta)`
IV.10	SrvSubasta → RepoOferta	`obtenerOfertaMayor(idSubasta)`
IV.11	SrvSubasta → RepoSubasta	`actualizarVersionSiAbiertaYEnPlazo(idSubasta, version)`
IV.12a	SrvSubasta → RepoOferta	`guardarOferta(idSubasta, cliente, valor)` a: filasAfectadas = 1
IV.13a	SrvSubasta → RepoOutboxSubasta	`guardar(CambioOfertaMayor)`
IV.12b	SrvSubasta → WSSubasta	`ofertaNoRegistrada(CONFLICTO_CONCURRENTE)` b: filasAfectadas = 0
IV.14	WorkerOutboxSubasta → RepoOutboxSubasta	`obtenerPendientes(tamañoLote)`
IV.15	WorkerOutboxSubasta → q.cambioOfertaMayor	`publicar(CambioOfertaMayor)` AMQP
IV.16	q.cambioOfertaMayor → ListenerEventoSubasta	`entregar(evento)`
IV.17	ListenerEventoSubasta → SrvSuscripcion	`procesarEvento(evento)`
IV.18	SrvSuscripcion → RepoSuscripcion	`yaProcesado(idEvento)` · `buscarSuscriptores()`
IV.19	SrvSuscripcion → RepoOutboxSuscripcion	`guardar(Notificacion)`
IV.20	WorkerOutboxSuscripcion → RepoOutboxSuscripcion	`obtenerPendientes(tamañoLote)`
IV.21	WorkerOutboxSuscripcion → q.notificacion	`publicar(Notificacion)` → IX

V — Cerrar subasta y crear compra

DS09, DS12

Verde azulado · RGB 0, 128, 128

V.1	TimerCierreSubasta → SrvSubasta	`cerrarSubastasVencidas()`

V.2	SrvSubasta → RepoSubasta	`buscarVencidas() · tomarParaCierre(idSubasta)`
V.3	SrvSubasta → RepoOferta	`obtenerOfertaMayor(idSubasta) · marcarResultado(...)`
V.4a	SrvSubasta → RepoSubasta	`adjudicar(idSubasta, cedulaGanador, valor)` a: hay ofertas
V.4b	SrvSubasta → RepoSubasta	`declararDesierta(idSubasta)` b: sin ofertas
V.5	SrvSubasta → RepoOutboxSubasta	`guardar(SubastaCerrada)`
V.6	WorkerOutboxSubasta → RepoOutboxSubasta	`obtenerPendientes(tamañoLote)`
V.7	WorkerOutboxSubasta → q.subastaCerrada	`publicar(SubastaCerrada)` AMQP
La cola entrega el evento a UNA réplica: a = réplica 1 (Máquina 2), b = réplica 2 (Máquina 3)		
V.8a/b	q.subastaCerrada → ListenerSubastaCerrada	`entregar(evento)`
V.9a/b	ListenerSubastaCerrada → SrvCompraSaga	`iniciarSaga(evento)`
V.10a/b	SrvCompraSaga → RepoCompra	`existePorSubasta(idSubasta) · crear(Compra)`
V.11a/b	SrvCompraSaga → RepoOutboxCompra	`guardar(NotificacionGanador)`
V.12a/b	WorkerOutboxCompra → RepoOutboxCompra	`obtenerPendientes(tamañoLote)`
V.13a/b	WorkerOutboxCompra → q.notificacion	`publicar(NotificacionGanador)` → IX

VI Iniciar compra y validar OFAC

DS05-A, DS05-B, DS13, DS14

Mostaza · RGB 204, 153, 0

VI.1	Browser → PantallaWebSubasta	`consultarMisCompras() · seleccionarCompra(numeroCompra)`
VI.2	PantallaWebSubasta → ProxyWSSubastaWeb	`iniciarCompra(numeroCompra, token)`
VI.2m	PantallaMobileSubasta → ProxyWSSubastaMobile	`iniciarCompra(numeroCompra, token)`
VI.3	ProxyWSSubastaWeb → Nginx	`POST /api/compras/{numero}/iniciar`
VI.3m	ProxyWSSubastaMobile → Nginx	`POST /api/compras/{numero}/iniciar`
VI.4	Nginx → WSGateway	`iniciarCompra(numeroCompra, token)`
VI.5	WSGateway → SrvSeguridadToken	`validarYAutorizar(solicitud)`
VI.6	SrvSeguridadToken → SrvRuteo	`enrutarInicioCompra(numeroCompra, cliente)`
VI.7	SrvRuteo → Nginx	`iniciarCompra (upstream msCompra, least_conn)`
VI.8	Nginx → WSCompra	`iniciarCompra(numeroCompra, cliente)` r1 y r2
VI.9	WSCompra → SrvCompraSaga	`iniciarCompra(numeroCompra, cliente)` r1 y r2
VI.10	SrvCompraSaga → RepoCompra	`obtener(numeroCompra) · cambiarEstado(EN_VALIDACION)` r1 y r2
VI.11	TimerValidacionOFAC → SrvCompraSaga	`validarComprasPendientes()` r1 y r2
VI.12	SrvCompraSaga → RepoCompra	`tomarPendientesValidacion(tamañoLote)` r1 y r2
VI.13	SrvCompraSaga → IntegraValidacion	`validar(numeroCompra, cedula)` r1 y r2
VI.14	IntegraValidacion → WSValidacion	`validar(numeroCompra, cedula)` r1 y r2 HTTP/REST
VI.15	WSValidacion → SrvValidarOFAC	`validar(numeroCompra, cedula)`
VI.16	SrvValidarOFAC → RepoValidar	`buscarResultado(numeroCompra)`
VI.17	SrvValidarOFAC → IntegraOFAC	`consultar(cedula)`
VI.18	IntegraOFAC → WSOfac	`validarCedula(cedula)` HTTP/REST
VI.19	WSOfac → SrvValidacionOFAC	`validarCedula(cedula)`
VI.20	SrvValidacionOFAC → RepoOFAC	`consultarReporte(cedula)`
VI.21	SrvValidarOFAC → RepoValidar	`guardarResultado(numeroCompra, estado)`
VI.22a	SrvCompraSaga → RepoCompra	`cambiarEstado(ESPERANDO_PAGO)` r1 y r2 a: APROBADO
VI.23a	SrvCompraSaga → RepoOutboxCompra	`guardar(PagoSolicitud)` r1 y r2 → VII
VI.22b	SrvCompraSaga → RepoCompra	`cambiarEstado(CANCELADA, "RECHAZADO OFAC")` r1 y r2 b: RECHAZADO
VI.23b	SrvCompraSaga → RepoOutboxCompra	`guardar(NotificacionCancelacion)` r1 y r2 → IX
VI.22c	SrvCompraSaga → RepoCompra	`registrarIntento(intento + 1)` r1 y r2 c: sin respuesta
VI.23c	SrvCompraSaga → RepoOutboxCompra	`cambiarEstado(EN_REVISION) · guardar(NotificacionDemora)` r1 y r2

VII — Pagar en YoPago (.NET)

DS15, DS06-A, DS06-B, DS16, DS17 — Azul oscuro · RGB 0, 0, 160

VII.1	WorkerOutboxCompra → RepoOutboxCompra	`obtenerPendientes(tamañoLote)` r1 y r2
VII.2	WorkerOutboxCompra → q.pagoSolicitud	`publicar(PagoSolicitud)` r1 y r2 · AMQP · Java → .NET
VII.3	q.pagoSolicitud → ListenerPagoSolicitud	`entregar(solicitud)`
VII.4	ListenerPagoSolicitud → SrvPago	`prepararPago(numeroCompra, cedula, valorOfertado)`
VII.5	SrvPago → RepoPago	`existe(numeroCompra)`
VII.6	SrvPago → IntegraYoPago	`registrarPendiente(cedula, numeroCompra, valorTotal)`
VII.7	IntegraYoPago → WSCompraPendiente	`registrarCompraPendiente(...)` HTTP/REST
VII.8	WSCompraPendiente → SrvYoPago	`registrarCompraPendiente(...)`
VII.9	SrvYoPago → RepoYoPago	`guardarPendiente(cedula, numeroCompra, valorTotal)`
VII.10	SrvPago → RepoPago	`guardar(Pago)`
VII.11	SrvPago → RepoOutboxPago	`guardar(NotificacionPago)`
VII.12	WorkerOutboxPago → RepoOutboxPago	`obtenerPendientes(tamañoLote)`
VII.13	WorkerOutboxPago → q.notificacion	`publicar(NotificacionPago)` → IX
VII.14	Browser → PantallaWebYoPago	`consultarCompraPendiente(numeroCompra)` · `confirmarPago(...)`
VII.15	PantallaWebYoPago → ProxyWSYoPagoWeb	`consultarCompraPendiente` · `pagar(numeroCompra, datosPago)`
VII.16	ProxyWSYoPagoWeb → WSCompraPendiente	`consultarCompraPendiente` · `pagar(...)` HTTP
VII.17	WSCompraPendiente → SrvYoPago	`pagar(numeroCompra, datosPago)`
VII.18	SrvYoPago → RepoYoPago	`buscarPendiente` · `registrarPagoConfirmado(...)`
VII.19	SrvYoPago → IntegraSSO	`reportarPago(cedula, numeroCompra, valorPagado, credencial)`
VII.20	IntegraSSO → Nginx	`POST /api/pagos/reporte` HTTPS
VII.21	Nginx → WSGateway	`reportarPago(cedula, numeroCompra, valorPagado, credencial)`
VII.22	WSGateway → SrvSeguridadToken	`validarYAutorizar(solicitudReportePago)`
VII.23	SrvSeguridadToken → SrvRuteo	`enrutarReportePago(cedula, numeroCompra, valorPagado)`
VII.24	SrvRuteo → WSPagoYoPago	`reportarPago(cedula, numeroCompra, valorPagado)` Java → .NET
VII.25	WSPagoYoPago → SrvPago	`conciliarPago(cedula, numeroCompra, valorPagado)`
VII.26	SrvPago → RepoPago	`obtenerPorCompra` · `confirmarSiPendiente(...)`
VII.27	SrvPago → RepoOutboxPago	`guardar(PagoConfirmado)`
VII.28	WorkerOutboxPago → RepoOutboxPago	`obtenerPendientes(tamañoLote)`
VII.29	WorkerOutboxPago → q.pagoConfirmado	`publicar(PagoConfirmado)` AMQP · .NET → Java
VII.30	q.pagoConfirmado → ListenerPago	`entregar(PagoConfirmado)` r1 y r2
VII.31	ListenerPago → SrvCompraSaga	`procesarPago(numeroCompra, referenciaPago)` r1 y r2
VII.32a	SrvCompraSaga → RepoCompra	`cambiarEstadoSiActual(ESPERANDO_PAGO → PAGO_CONFIRMADO)` r1 y r2 · a: aprobado
VII.33a	SrvCompraSaga → RepoOutboxCompra	`guardar(DespachoSolicitud)` r1 y r2 → VIII
VII.32b	SrvCompraSaga → SrvCompraSaga	`compensar(numeroCompra, "pago rechazado")` r1 y r2 → X · b: rechazado

VIII — Despachar con SIN y SEN

DS18, DS19, DS20, DS21-A, DS21-B, DS21-C — Oliva · RGB 128, 128, 0

VIII.1	WorkerOutboxCompra → RepoOutboxCompra	`obtenerPendientes(tamañoLote)` r1 y r2
VIII.2	WorkerOutboxCompra → q.despachoSolicitud	`publicar(DespachoSolicitud)` r1 y r2 · AMQP
VIII.3	q.despachoSolicitud → ListenerDespachoSolicitud	`entregar(evento)`
VIII.4	ListenerDespachoSolicitud → SrvDespacho	`solicitarDespacho(numeroCompra, idBien, direccion)`
VIII.5	SrvDespacho → RepoDespacho	`existePorCompra` · `crear(Despacho)`
VIII.6	SrvDespacho → RepoOutboxDespacho	`guardar(SolicitudInventario)` · `guardar(SolicitudEnvio)`
VIII.7	WorkerOutboxDespacho → RepoOutboxDespacho	`obtenerPendientes(tamañoLote)`
VIII.8a	WorkerOutboxDespacho → q.inventarioSolicitud	`publicar(SolicitudInventario)` a: SIN
VIII.9a	q.inventarioSolicitud → ListenerInventario	`entregar(evento)`

VIII.10a	ListenerInventario → SrvInventario	`registrarSalidaInventario(numeroCompra, idBien)`
VIII.11a	SrvInventario → RepoInventario	`existePorCompra · registrarSalida(...)`
VIII.12a	SrvInventario → RepoOutboxSIN	`guardar(InventarioConfirmado)`
VIII.13a	WorkerOutboxSIN → RepoOutboxSIN	`obtenerPendientes(tamañoLote)`
VIII.14a	WorkerOutboxSIN → q.inventarioConfirmacion	`publicar(InventarioConfirmado)`
VIII.15a	q.inventarioConfirmacion → ListenerConfInventario	`entregar(evento)`
VIII.16a	ListenerConfInventario → SrvDespacho	`confirmarInventario(numeroCompra)`
VIII.8b	WorkerOutboxDespacho → q.envioSolicitud	`publicar(SolicitudEnvio)` b: SEN
VIII.9b	q.envioSolicitud → ListenerEnvio	`entregar(evento)`
VIII.10b	ListenerEnvio → SrvEnvio	`programarEnvio(numeroCompra, idBien, direccion)`
VIII.11b	SrvEnvio → RepoEnvio	`existePorCompra · programar(...)`
VIII.12b	SrvEnvio → RepoOutboxSEN	`guardar(EnvioConfirmado)`
VIII.13b	WorkerOutboxSEN → RepoOutboxSEN	`obtenerPendientes(tamañoLote)`
VIII.14b	WorkerOutboxSEN → q.envioConfirmacion	`publicar(EnvioConfirmado)`
VIII.15b	q.envioConfirmacion → ListenerConfEnvio	`entregar(evento)`
VIII.16b	ListenerConfEnvio → SrvDespacho	`confirmarEnvio(numeroCompra)`
VIII.17	SrvDespacho → RepoDespacho	`marcarInventarioConfirmado / marcarEnvioConfirmado`
VIII.18	SrvDespacho → RepoOutboxDespacho	`guardar(DespachoConfirmado) (solo si ambos confirmados)`
VIII.19	WorkerOutboxDespacho → RepoOutboxDespacho	`obtenerPendientes(tamañoLote)`
VIII.20	WorkerOutboxDespacho → q.despachoConfirmado	`publicar(DespachoConfirmado)`
VIII.21	q.despachoConfirmado → ListenerConfDespacho	`entregar(evento)` r1 y r2
VIII.22	ListenerConfDespacho → SrvCompraSaga	`confirmarDespacho(numeroCompra)` r1 y r2
VIII.23	SrvCompraSaga → RepoCompra	`cambiarEstadoSiActual(PAGO_CONFIRMADO → TERMINADA)` r1 y r2
VIII.24	SrvCompraSaga → RepoOutboxCompra	`guardar(NotificacionCompraFinalizada)` r1 y r2 → IX

IX — Notificar al cliente

DS22 (llega desde III, IV, V, VI, VII, VIII y X)

Azul · RGB 0, 112, 192

IX.1	q.notificacion → ListenerNotificar	`entregar(evento)`
IX.2	ListenerNotificar → SrvNotificar	`notificar(evento)`
IX.3	SrvNotificar → RepoNotificar	`yaEnviada(idEvento)`
IX.4	SrvNotificar → IntegraMail	`enviarCorreo(destinatario, asunto, cuerpo)`
IX.5	IntegraMail → CORREO	`enviar(mensaje)` SMTP
IX.6	SrvNotificar → IntegraSMS	`enviarSMS(numero, texto) (solo si el evento lo pide)`
IX.7	IntegraSMS → SMS GATEWAY	`enviar(numero, texto)` HTTP
IX.8	SrvNotificar → RepoNotificar	`registrarEnviada(idEvento, canal, resultado)`

X — Compensar la compra

DS23, DS24-A, DS24-B, DS24-C (viene de VII.32b)

Café · RGB 128, 64, 0

X.1	SrvCompraSaga → RepoCompra	`cambiarEstado(→ CANCELADA, motivo)` r1 y r2 pago rechazado
X.2	SrvCompraSaga → RepoOutboxCompra	`guardar(BienLiberado)` r1 y r2
X.3	WorkerOutboxCompra → RepoOutboxCompra	`obtenerPendientes(tamañoLote)` r1 y r2
X.4	WorkerOutboxCompra → q.LiberarBienSubasta	`publicar(BienLiberado)` r1 y r2 AMQP
X.5a	q.LiberarBienSubasta → ListenerLiberarBienSub	`entregar(BienLiberado)` a: MSSubasta
X.6a	ListenerLiberarBienSub → SrvSubasta	`liberarBien(idSubasta, motivo)`
X.7a	SrvSubasta → RepoSubasta	`liberarSiAdjudicada(idSubasta)`
X.8a	SrvSubasta → RepoOutboxSubasta	`guardar(BienLiberadoConfirmado)`
X.5b	q.LiberarBienSubasta → ListenerLiberarBienCat	`entregar(BienLiberado)` b: MSCatalogo
X.6b	ListenerLiberarBienCat → SrvCatalogo	`liberarBien(idSubasta, motivo)`
X.7b	SrvCatalogo → RepoCatalogo	`liberarSiAdjudicada(idSubasta)`
X.8b	SrvCatalogo → RepoOutboxCatalogo	`guardar(BienLiberadoConfirmado)`

X.5c	q.LiberarBienSubasta → ListenerLiberarBienSus	`entregar(BienLiberado)` c: MSSuscripcion
X.6c	ListenerLiberarBienSus → SrvSuscripcion	`liberarBien(idSubasta, motivo)`
X.7c	SrvSuscripcion → RepoSuscripcion	`liberarSiAdjudicada(idSubasta)`
X.8c	SrvSuscripcion → RepoOutboxSuscripcion	`guardar(BienLiberadoConfirmado)`
X.9	SrvCompraSaga → RepoCompra	`cambiarEstadoSiActual(PAGO_CONFIRMADO → EN_REVISION)` r1 y r2 despacho agotado
X.10	SrvCompraSaga → RepoOutboxCompra	`guardar(NotificacionRevisionManual) (no libera el bien)` r1 y r2
X.11	WorkerOutboxCompra → q.notificacion	`publicar(NotificacionRevisionManual)` r1 y r2 → IX